Projet : Tableau de bord de véhicule avec CAN

Hassan Jradi

Ph.D. in Telecoms and Security, RAN Security System Manager at Ericsson

hassan.jradi@intervenants.efrei.net

Réseaux de Communication Temps-Réel (ST2RCTR)

Efrei Paris 2025-2026

Objectif

Le but de ce projet est de développer une Application interactive sous MATLAB, à l'aide de MATLAB App Designer et de la Vehicle Network Toolbox (VNT), capable de simuler un tableau de bord de véhicule.

Contexte

Dans les véhicules modernes, les différents calculateurs électroniques (ECUs) communiquent entre eux via un réseau CAN (Controller Area Network). Le tableau de bord reçoit en permanence des messages de ces ECUs pour informer le conducteur de l'état du véhicule. Ces messages contiennent des données telles que la vitesse, le régime moteur, la température du moteur, le niveau de carburant, etc.

Un système de surveillance est également intégré pour détecter les situations critiques (surchauffe, niveau bas de carburant, régime moteur excessif, etc.). Lorsqu'un paramètre franchit un seuil critique, des voyants lumineux s'activent pour alerter le conducteur.

Ce projet vise à reproduire ce comportement à travers une application MATLAB, en simulant à la fois la réception des données CAN et l'affichage dynamique des indicateurs du tableau de bord.

Application

Interface Graphique

L'interface graphique de l'application est illustrée dans la Figure 1. Elle se compose des éléments suivants :

- Switch pour démarrer la voiture (et les composants électroniques)
- Jauge de Vitesse
- Jauge de Compte-tours
- Jauge de Niveau Carburant
- Jauge de Température
- Lampe pour l'indicateur « STOP » (couleur grise et rouge)
- Lampe pour le voyant Moteur (couleur grise, jaune et rouge)
- Lampe pour le voyant Carburant (couleur grise, jaune et rouge)
- Lampe pour l'état de Radiateur (couleur grise et jaune)
- Lampe pour l'état de la voiture (couleur grise et verte)

Pour les jauges, il faut respecter les valeurs limites visualisées et les couleurs des zones critiques (jaune et rouge), et il est recommandé de respecter l'espacement entre les tics mineurs et majeurs.

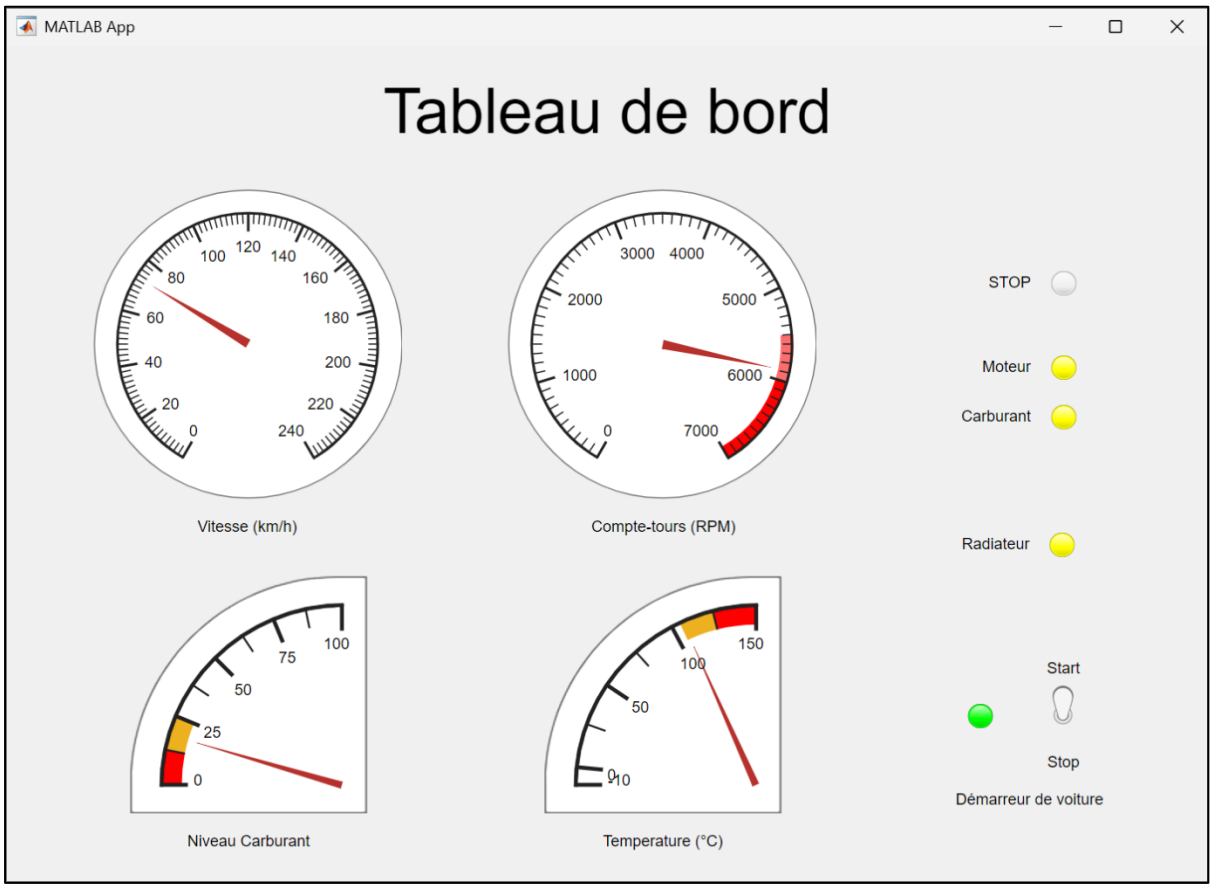


Figure 1: Interface graphique de l'application.

Fonction du script generateMsgsDb.m

Il ne faut rien changer dans ce script.

Le script « generateMsgDB.m » fourni sert à simuler le rôle des composants électroniques de la véhicule qui collectent des données et les envoient sur le bus CAN. Il exploite le canal « MathWorks Virtual 1 – Channel 1 » pour diffuser, toutes les secondes, des trames définies dans la base DBC « tableau_de_bord.dbc ».

Utiliser le chanel 2 car erreur avec chanel 1

La simulation s'étend sur une quinzaine de secondes ; à chaque itération, les valeurs des signaux sont ajustées pour reproduire une évolution réaliste :

- Le signal NiveauCarburant commence avec une valeur initiale de 48 et diminue avec le temps.
- La valeur du signal VitesseVehicule est variable avec une moyenne de 50 km/h et une variance de 30 km/h.
- La valeur du signal MoteurRPM est variable avec une moyenne de 4000 RPM et une variance de 2500 RPM.
- Le signal MoteurTemp commence avec une valeur initiale de 70 et a une tendance croissante avec le temps.

Logique (Comportement)

L'application démarre lorsque l'interrupteur passe à l'état « Start », c'est-à-dire que l'application démarre le canal de réception *rxCh* sur « MathWorks Virtual 1 - Channel 2 ». La couleur de la lampe de la voiture doit passer au vert. L'application est maintenant prête à recevoir les messages envoyés sur le bus CAN (par *txCH* avec generateMsgsDb.m).

Chaque fois qu'un message est reçu, et en utilisant le concept de **Callback**, les valeurs des signaux associées doivent être affichées sur les jauges correspondantes, et en outre, les différentes voyants (lampes) doivent s'allumer comme suit :

Voyant	Signal	Gris	Jaune	Rouge
Moteur	Compte-tours	[0, 5500[	[5500, 6000[	[6000, 7000]
Carburant	Niveau Carburant	[25, 100]	[12.5, 25[	[0, 12.5[
Moteur	Température	[-10, 105[	[105, 125[	[125, 150]

L'indicateur « STOP » s'allume en rouge si le voyant du moteur est rouge.
La lampe du radiateur s'allume en jaune pour indiquer son fonctionnement si la température du moteur est supérieure à 70°C.

Le fait de basculer l'interrupteur en position « Stop » éteint tous les voyants, remet les jauges à zéro et arrête le canal *rxCh*.

Fichier DBC

Le fichier « tableau_de_bord.dbc » doit contenir la définition des deux structures de messages CAN utilisées par l'application et les calculateurs, dont la première est donnée dans le fichier.

Il reste à définir la structure du deuxième message décrit ci-dessous :

- Identifiant du message : 100
- Nom du message : MoteurData
- Longueur : 4 octets

Ce deuxième message contient les deux signaux suivants :

Signal	**StartBit**	**Longueur**	**Type**	**Facteur d'échelle** **Offset**	**Range**	**Unité**
MoteurRPM	0	2 octets	Non-signé	0.125 0	[0,7000]	RPM
MoteurTemp	16	2 octets	Signé	1 0	[-10,150]	°C

Lancement

Pour lancer la simulation, vous devez d'abord démarrer l'application, puis lancer le script generateMsgsDb.m. Vous devez alors voir les jauges changer et les voyants correspondants s'allumer avec la couleur correspondante.

Grille d'évaluation (/100pts)

Critère	Points	Indicateurs observables
1. Fonctionnalités techniques	40	- Démarrage/arrêt correct du canal CAN (rxCh). - Réception et décodage en temps réel des messages. - Mise à jour fiable des quatre jauges. - Activation des voyants selon les plages imposées, y compris STOP et radiateur. - Remise à zéro propre après arrêt.
2. Interface graphique	20	- Présence de tous les éléments de l'interface graphique. - Respect des limites et couleurs critiques des jauges. - Positionnement clair et lisible des éléments.
3. Logique de contrôle	15	- Gestion d'événements via callback sans retards ni blocages.
4. Conformité du fichier DBC	10	- Message MoteurData correctement défini. - Signaux MoteurRPM et MoteurTemp correctement définis.
5. Qualité du code	15	- Code clair et structuré (fonctions, variables, noms explicites). - Absence de duplication inutile. - Bonne gestion des objets CAN.

Modalités de restitution

Date limite	26 juillet 2026 (inclus)
Plateforme	Moodle
Format	Deux fichiers : - TableauBordApp.mlapp - tableau_de_bord.dbc